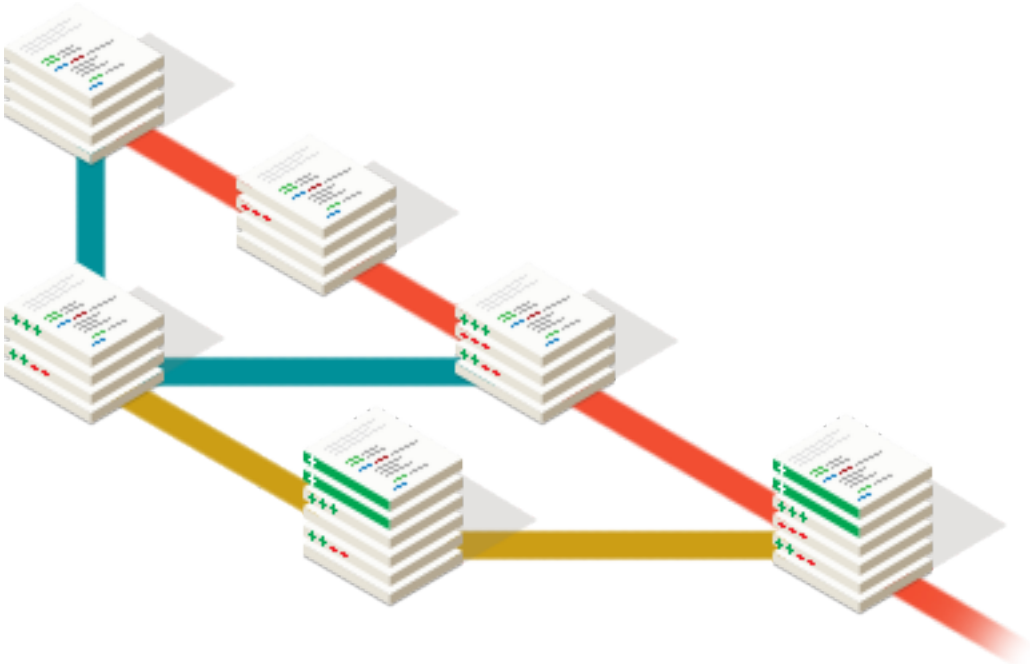


Git: Day 5

DIFFS

What is a diff

A compact summary of the differences between two items.

Diffs can help you make well-structured commits during your normal development process.

Forms of the git diff

There are three basic sources to use git diff:

- Any object (commit, branch name, tag, etc) anywhere within the entire commit graph.

- Your working directory.

- The index

Forms of the git diff

$ git diff

Shows the difference between your working directory and the index.

Exposes what is dirty in your working directory.

Does not reveal differences between what's in your index and what is permanently stored in the repository.

Forms of the git diff

$ git diff <commit>

Summarizes differences between your working directory and the given commit.

<commit> can be HEAD or a particular branch name.

Forms of the git diff

$ git diff --cached <commit>

Shows the differences between index and the given commit.

Default <commit> value is HEAD that shows you how your next commit will alter the current branch.

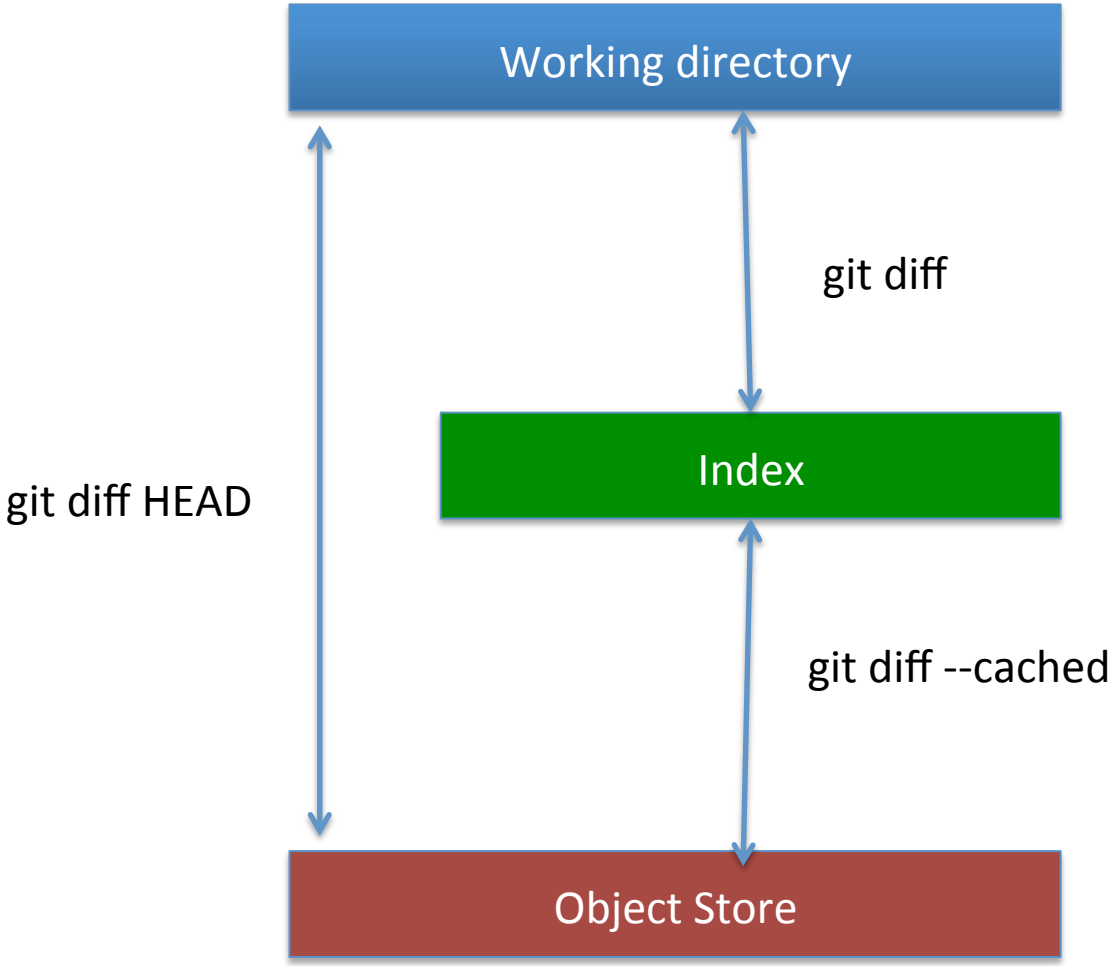
Forms of the git diff

$ git diff <commit1> <commit2>
$ git diff <commit1>..<commit2>

Differences between the two commits.

This command ignores the index and working directory.

Forms of the git diff



Forms of the git diff

$ git diff <commit1> <commit2> <file>
$ git diff <commit1>..<commit2> <file>

The file differences between the two commits.

This command ignores the index and working directory.

Options

-M: Detects renames and generates a simplified output.

-w: Compare lines without considering changes in whitespace as significant

--stat: Adds statistics about the difference between any two tree states.

--color: Colorizes the output

ADVANCED CONFIGURATION

Alias

Aliases can avoid typing the same command over and over again.

Aliases were added in Git 1.4.0

Alias

Configuration:

- Edit .git/config

  [alias]

  ci = commit

- $HOME/.gitconfig: Available everywhere

- $ git config –global alias.ci commit

Git Blame

Tells you who last modified each line of a file and which commit made the change:

```
$ git blame <file>
```

Repository Types

There are two different types of Git repos:

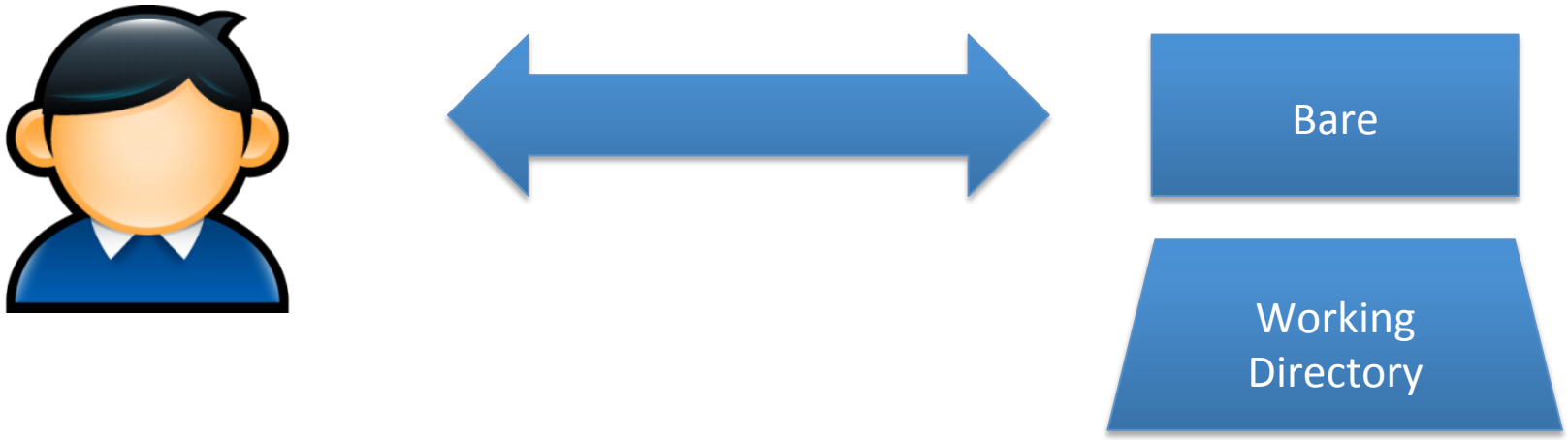
- **Bare:** Has no working directory. Not be used for normal development. No direct commits.

- **Development:** Typical repository. Maintains current branch, provides checked-out copy of the current branch in a **working directory**

Repository Types

Bare repo is crucial for collaborative development.

Other developers clone and fetch from the bare repository and push updates to it



Other developer

My Computer

Bare repo creation

```
$ git init <repository> --bare
```

Best Practice:

A published repository should be bare

Preguntas

Israel Alcázar
israelalcazar@gmail.com
@ialcazar